

Data Science (COL2010): Homework 1

Module 1: Database Systems and SQL Querying

Submission: `solutions.py` only. Run `setup.py` before testing.

Learning Outcomes

After completing this homework, you should be able to write raw SQL for selection, projection, joins, aggregation, subqueries, set operations, NULL handling, string manipulation, window functions, and recursive common table expressions. You should also be able to execute SQL programmatically using Python's `sqlite3` and `pandas` interfaces.

NOTE - Submission and autograder instructions will be provided soon. For now you can start attempting the tasks.

Setup and Workflow

Run the setup script once:

```
python setup.py
```

This creates `logistics_network.db` (the SQLite database to be queried) and `hw1_query_inputs.json` (the setup-generated input parameters for every function).

Do not submit the generated database. The autograder will create its own copy using the same schema.

Files Provided

- `setup.py`: creates and populates the SQLite database, then creates `hw1_query_inputs.json` containing all function arguments.
- `skeleton-solutions.py`: function signatures and helper execution wrappers. Rename this file as `solutions.py` when you submit.

Important Rule for this Module

Every task must be solved by writing **raw SQL** inside the specified Python function and firing it programmatically. You may use `sqlite3.connect`, `cursor.execute`, and `pd.read_sql_query`. Do **not** replace the SQL task by manually reconstructing the answer in pandas.

How Function Inputs Are Generated

Unlike a conventional written SQL assignment, each function is parameterized and the inputs are generated by `setup.py`. Running

```
python setup.py
```

creates two artifacts:

1. `logistics_network.db`: the SQLite database that every SQL query is fired against.
2. `hw1_query_inputs.json`: the exact keyword arguments for each task.

Students should not invent the parameters manually. They must load them from the JSON file. Rerunning `setup.py` generates a set of new random parameters everytime which can be used for testing on different inputs.

The autograder will use the same idea: it will run `setup.py`, load `hw1_query_inputs.json`, and call each function with the corresponding generated arguments. For example:

- Task 5 receives a real `sku_id` and `node_id` pair sampled from `node_inventory`.
- Task 6 receives a shipment that actually has manifest rows.
- Tasks 28–29 and 34 receive node endpoints with valid multi-hop paths.

Tasks

Tasks	Setup-generated Inputs
1	No argument beyond the returned SQL string.
2–4	Capacity bounds, shipment status flag, and risk baseline.
5–6	Valid SKU–node candidate pair and valid shipment identifier.
7–25	Database path plus query-specific thresholds, date bounds, regex pattern, or source-type list where required.
26–35	Database path plus recursive-query parameters such as employee id, manager id, route endpoints, SKU, and blocked node id.

Generated Input File Format

The JSON file has one dictionary entry per task. The keys are named `task_01`, `task_02`, ..., `task_35`. Each value is directly unpacked into the corresponding function call.

```
{
  "database": {
    "db_path": "logistics_network.db"
  },
  "task_02": {
    "db_path": "logistics_network.db",
    "min_cap": 50000.0,
    "max_cap": 120000.0
  },
  "task_05": {
    "db_path": "logistics_network.db",
    "sku_id": "<setup-generated-valid-sku>",
    "node_id": "<setup-generated-valid-node-id>"
  }
}
```

Therefore, the function signature remains fixed, but the actual argument values are supplied by the setup code.

Database Schema

The generated database has the following tables.

Table	Meaning
nodes	Logistics hubs/warehouses/ports/rail yards with geographic coordinates, region, type, and capacity.
routes	Directed transportation links between source and destination nodes with mode, distance, base cost, quarter, and active flag.
inventory_items	Product catalogue containing SKU, product name, category, unit mass, unit volume, and hazardous-material flag.
node_inventory	Junction table storing quantity of each item at each node. Primary key: (node_id, item_id).
shipments	Shipment records over routes with carrier, departure/arrival timestamps, status, and freight cost.
shipment_manifests	Junction table listing which items and quantities occur in each shipment.
organizational_hierarchy	Employee hierarchy with node assignment, manager pointer, role title, and salary.
route_edges	Simplified graph edge table derived from routes, useful for recursive network traversal tasks.
bom_components	Bill-of-materials relation linking a master item to component items and quantities.

Representative Schema Fragments

```
CREATE TABLE nodes (  
  node_id INTEGER PRIMARY KEY,  
  node_name TEXT NOT NULL,  
  city TEXT NOT NULL,  
  region TEXT NOT NULL,  
  node_type TEXT NOT NULL,  
  latitude REAL NOT NULL,  
  longitude REAL NOT NULL,  
  capacity_cubic_meters REAL NOT NULL  
);  
  
CREATE TABLE routes (  
  route_id INTEGER PRIMARY KEY,  
  source_node_id INTEGER NOT NULL REFERENCES nodes(node_id),  
  dest_node_id INTEGER NOT NULL REFERENCES nodes(node_id),  
  transport_mode TEXT NOT NULL,  
  distance_km REAL NOT NULL,  
  base_cost_usd REAL NOT NULL,  
  seasonal_quarter TEXT NOT NULL,  
  route_active INTEGER NOT NULL  
);
```

Marking Scheme

Part I contains Tasks 1–25 and is worth 70 marks. Part II contains Tasks 26–35 and is worth 30 marks. Exploration components are not autograded unless explicitly requested by the instructor, but they are included to guide deeper experimentation.

Part I: SQL Foundations and Medium Queries

Task 1: Schema Integrity Definition Verification

Description. Write a CREATE TABLE statement for `operational_audit_logs` with the following fields:

- `audit_id`
- `event_time`
- `actor_id`
- `action_code`
- `entity_name`
- `entity_key`
- `payload_json`

`audit_id` must be an AUTOINCREMENT primary key. Use NOT NULL fields where appropriate, and a composite UNIQUE constraint (to prevent the same activity from being logged twice). Return the raw SQL string.

Function signature.

```
build_operational_audit_schema() -> str
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
SCHEMA_VERIFICATION_IS_VALID: TRUE
```

Free to Explore

Create an index on `action_code` and compare insertion/query timing.

Task 2: Dynamic Range Capacity Isolation Filter

Description. Write SQL to select node_id, node_name, and capacity_cubic_meters for nodes whose capacity lies between the given bounds.

Function signature.

```
extract_bounded_capacity_nodes(db_path: str, min_cap: float, max_cap: float) -> pd.  
    DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
FILTERED_RECORDS_COUNT: XXXX  
MEAN_EXTRACTED_CAPACITY: XXXX.XX
```

Free to Explore

Change the capacity interval width and plot how selectivity changes.

Task 3: Transit Duration Sorting Engine

Description. Use julianday arithmetic (read about it) to compute transit duration in hours for shipments with the given status and non-null arrivals. Sort the rows in descending order of transit duration and break ties in increasing order of shipment_id.

Function signature.

```
identify_highest_transit_delays(db_path: str, status_flag: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
LONGEST_TRANSIT_HOURS: XXX.XX  
CARRIER_IDENTITY_STRING: XXX_XXX
```

Free to Explore

Compare transit-time distributions across Delivered and Delayed shipments.

Task 4: Conditional Cost Risk Classification

Description. Use CASE WHEN to label routes as High-Cost-Tier or Standard-Cost-Tier by applying a baseline_limit on base_cost_usd.

Function signature.

```
append_freight_risk_profiles(db_path: str, baseline_limit: float) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
HIGH_RISK_ROUTE_COUNT: XXXX  
RATIO_TO_TOTAL_SYSTEM: X.XXXX
```

Free to Explore

Sweep baseline_limit and plot the high-risk fraction.

Task 5: Candidate Compound Match Verification

Description. Use a join between node_inventory and inventory_items to test if a SKU exists at a node.

Function signature.

```
verify_candidate_presence(db_path: str, sku_id: str, node_id: int) -> bool
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
CANDIDATE_MATCH_CONFIRMED: <TRUE/FALSE>
```

Free to Explore

Try valid and invalid SKU/node combinations and explain why composite keys matter.

Task 6: Multi-Layer Inner Join Structural Weight Evaluation

Description. Join shipment_manifests with inventory_items and compute total manifest mass as quantity times unit weight.

Function signature.

```
calculate_shipment_manifest_mass(db_path: str, shipment_id: int) -> float
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
COMPUTED_MANIFEST_MASS_KG: XXXXXX.XX
```

Free to Explore

Compare mass distributions for low-cost and high-cost shipments.

Task 7: Left Outer Join Orphan Coverage Analytics

Description. Use a LEFT JOIN to find inventory items that never appear in shipment_manifests.

Function signature.

```
isolate_unreferenced_inventory_items(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
ORPHAN_INVENTORY_ROW_COUNT: XXXX
```

Free to Explore

Insert a new unused SKU and verify that the query detects it.

Task 8: Regular Expression String Match Execution

Description. Register a SQLite REGEXP function from Python and filter SKUs matching the given regular expression.

Function signature.

```
filter_sku_by_regex_pattern(db_path: str, exact_regex: str) -> list
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
REGEX_MATCHED_ELEMENTS_COUNT: XXX
```

Free to Explore

Design three regex patterns of increasing specificity and compare match counts.

Task 9: Multi-Column Ordered Sequence Isolation

Description. Sort carrier shipment rows by freight cost descending and then by status-derived delay rank.

Function signature.

```
sequence_carrier_allocations(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
TOP_CARRIER_ROW_HASH: XXX_XXX
```

Free to Explore

Change tie-breaking attributes and observe whether the first row changes.

Task 10: Correlated Inclusion via Nested Subquery

Description. Use a correlated subquery to find nodes whose total inventory quantity exceeds a threshold.

Function signature.

```
identify_high_density_warehouses(db_path: str, asset_threshold: int) -> list
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
HIGH_DENSITY_STATIONS_COUNT: XXX
```

Free to Explore

Sweep `asset_threshold` and plot number of qualifying hubs.

Task 11: Set Inclusion Operators Verification

Description. Use IN with a dynamic placeholder list to filter routes by the type of their source node.

Function signature.

```
filter_routes_by_source_inclusion(db_path: str, valid_source_types: list) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
MATCHED_INCLUSION_ROUTE_COUNT: XXXX
```

Free to Explore

Compare route-mode composition across different source node types.

Task 12: Group Aggregation with HAVING

Description. Group shipments by carrier, calculate the total number of shipment runs, total freight volume, and average freight cost per run for each carrier. Filter the results to include only carriers with at least a specified minimum number of runs using the HAVING clause, and sort the carriers by total freight volume in descending order.

Function signature.

```
aggregate_carrier_performance(db_path: str, minimum_runs: int) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
QUALIFIED_CARRIERS_COUNT: XX  
AGGREGATE_SYSTEM_VOLUME_USD: XXXXXXX.XX
```

Free to Explore

Vary minimum_runs and explain the stability of aggregate carrier rankings.

Task 13: Correlated Salary Threshold Evaluator

Description. Use a correlated subquery to find employees earning more than their assigned node average.

Function signature.

```
locate_outlier_salaries_by_hub(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
OUTLIER_SALARY_ROW_COUNT: XXX  
MAX_DEVIATION_METRIC: XXXXX.XX
```

Free to Explore

Repeat using region-level averages instead of node-level averages.

Task 14: Union Operator Active Inventory Items

Description. Return the unique `item_ids` that appear either in warehouse inventory or in shipment manifests. This identifies all inventory items that are currently stored at a node or are part of at least one shipment.

Function signature.

```
extract_active_item_ids(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
TOTAL_ACTIVE_ITEMS: XXXX
```

Free to Explore

Compare UNION with UNION ALL. Explain why an item appearing in both inventory and shipment manifests is returned only once when using UNION.

Task 15: Cross-Join Cartesian Space Evaluator

Description. Use CROSS JOIN to count all theoretical node-item allocation pairs.

Function signature.

```
compute_cartesian_density_space(db_path: str) -> float
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
TOTAL_CARTESIAN_MATRIX_ROWS: XXXXXXXX
```

Free to Explore

Estimate how this count scales as nodes and items grow.

Task 16: Intersect Boundaries Matrix Resolution

Description. Use INTERSECT to find shipments satisfying both departure and arrival time constraints.

Function signature.

```
isolate_overlapping_schedule_runs(db_path: str, start_bound: str, end_bound: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
INTERSECTED_SCHEDULE_COUNT: XXX
```

Free to Explore

Rewrite the same logic using AND and compare query plans.

Task 17: Manifest Weight Variance Computation

Description. Compute per-shipment manifest mass and then compute its population variance using SQL aggregation.

Function signature.

```
calculate_manifest_deviation_variance(db_path: str) -> float
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
MANIFEST_WEIGHT_VARIANCE_VALUE: XXXXXX.XXXX
```

Task 18: Multi-Tier Subquery Above-Average Inventory Nodes

Description. Use nested subqueries to identify the nodes whose total inventory quantity is greater than the average inventory quantity across all nodes.

Function signature.

```
identify_high_inventory_nodes(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
TOTAL_HIGH_INVENTORY_NODES: XXXX
```

Free to Explore

Modify the query to return only the top 10% of inventory nodes by total inventory quantity.

Task 19: Null Field Remapping with COALESCE

Description. Use COALESCE to replace missing arrival timestamps by a fixed fallback timestamp.

Function signature.

```
remap_missing_timestamps(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
RESOLVED_NON_NULL_COUNT: XXXXXX
```

Free to Explore

Compare COALESCE with filtering out NULL rows.

Task 20: Concatenated String Key Construction

Description. Build standardized barcodes by concatenating region, node_type, and zero-padded node_id.

Function signature.

```
build_standardized_barcodes(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
BARCODE_GENERATION_SUCCESS_COUNT: XXXXX
```

Free to Explore

Change the barcode layout and check for uniqueness.

Task 21: Metric Conversion Projection

Description. Convert route distance from kilometers to miles inside SQL.

Function signature.

```
project_metric_conversions(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
CONVERTED_TOTAL_SUM_MILES: XXXXXXXX.XX
```

Free to Explore

Add a nautical-mile conversion and compare totals.

Task 22: Duplicate SKU Trapping Analysis

Description. Group by sku and use HAVING COUNT(*) > 1 to detect duplicate structural identifiers.

Function signature.

```
isolate_duplicate_skus(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
DUPLICATED_SKU_REDUNDANCIES_COUNT: XXX
```

Free to Explore

Insert controlled duplicate rows and verify the count.

Task 23: Partitioned Window Extraction

Description. Use `ROW_NUMBER() OVER(PARTITION BY assigned_node_id ORDER BY salary_usd DESC)` to get top-3 salaries per node.

Function signature.

```
slice_top_salaries_per_node(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
TOTAL_WINDOW_ROWS_EXTRACTED: XXXX
```

Free to Explore

Compare `ROW_NUMBER`, `RANK`, and `DENSE_RANK` under ties.

Task 24: Route Intersection Evaluation

Description. Self-join routes to find trajectory intersections where one route destination is another route source.

Function signature.

```
cross_reference_route_intersections(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
INTERSECTING_TRAJECTORIES_COUNT: XXXX
```

Free to Explore

Restrict to active routes only and compare the count.

Task 25: Heavy Multi-Join Aggregate Window Filtering

Description. Join shipments, routes, and nodes; filter long-transit rows; group by hub; compute aggregate freight summaries.

Function signature.

```
evaluate_high_density_cost_centers(db_path: str) -> pd.DataFrame
```

SQL requirement. Write the core logic as a SQL query string and execute it programmatically using SQLite.

Target output format

```
CRITICAL_COST_NODES_IDENTIFIED: XXX
MAX_AGGREGATE_HUB_FREIGHT_USD: XXXX.XX
SYSTEM_WIDE_MEAN_VARIANCE: XX.XXXX
```

Free to Explore

Run EXPLAIN QUERY PLAN before and after adding indexes.

Part II: Recursive SQL and Graph Queries

Task 26: Ascending Management Chain Tracer

Description. Use a recursive CTE to walk from an employee up through manager.employee_id until the root manager.

Function signature.

```
trace_management_hierarchy_chain(db_path: str, start_employee_id: int) -> list
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
RECURSIVE_TRAVERSAL_DEPTH: XX
TOP_LEVEL_SUPERVISOR_VERIFIED: TRUE
```

Free to Explore

Add an explicit recursion-depth safety bound and explain why it matters.

Task 27: Downstream Subordinate Extractor

Description. Use a recursive CTE to find all employees transitively reporting to a manager.

Function signature.

```
extract_downstream_subordinates(db_path: str, manager_id: int) -> list
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
TOTAL_DOWNSTREAM_SUBORDINATES_COUNT: XXXX
```

Free to Explore

Compare immediate reports with full downstream reports.

Task 28: Multi-Leg Hub Connection Path Generator

Description. Use recursive route expansion to enumerate simple paths up to a bounded depth from origin to terminal.

Function signature.

```
generate_multi_leg_connections(db_path: str, origin_id: int, terminal_id: int) -> pd.  
DataFrame
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
TOTAL_VALID_TRANSIT_PATHS_FOUND: XXX
```

Free to Explore

Increase max depth and observe growth in path count.

Task 29: Shortest Cumulative Distance Route

Description. Use recursive path expansion and MIN(total_distance) to compute the shortest bounded path.

Function signature.

```
find_shortest_distance_route(db_path: str, origin_id: int, terminal_id: int) -> float
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
MINIMAL_PATH_DISTANCE_KM: XXXXX.XX
```

Free to Explore

Compare bounded recursive search with Dijkstra implemented in Python.

Task 30: Network Cycle Detection Scanner

Description. Use recursive route traversal with a visited path string to detect cycles.

Function signature.

```
scan_network_cyclic_dependencies(db_path: str) -> list
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
CYCLIC_PATH_ANOMALIES_DETECTED: XX
```

Free to Explore

Insert a known cycle and verify that the query reports it.

Task 31: Transitive Asset Flow Density Tracker

Description. Starting from nodes that contain a SKU, recursively collect nearby reachable nodes and sum inventory quantity.

Function signature.

```
evaluate_transitive_asset_flows(db_path: str, item_sku: str) -> int
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
TRANSITIVE_FLOW_DENSITY_TOTAL: XXXXXX
```

Free to Explore

Change the hop depth and plot total reachable asset quantity.

Task 32: Nested Bill-of-Materials Mass Multiplier

Description. Use recursive traversal over bom_components to compute aggregate master plus component mass.

Function signature.

```
compute_bom_packaging_mass(db_path: str, master_sku: str) -> float
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
AGGREGATE_BOM_COMPONENT_MASS_KG: XXXXX.XX
```

Free to Explore

Create a deeper BOM tree and verify multiplicative quantities.

Task 33: Reachability Under Node Failure

Description. Use recursive traversal while excluding a blocked node to test reachability.

Function signature.

```
evaluate_hub_reachability_matrix(db_path: str, source_node_id: int, target_node_id: int,  
    blocked_node_id: int) -> bool
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
TARGET_TERMINAL_REACHABLE: <TRUE/FALSE>
```

Free to Explore

Try several blocked nodes and identify critical hubs.

Task 34: Dynamic Multi-Hop Cost Accumulation

Description. Use recursive traversal to return paths and cumulative cost, sorted by total_cost.

Function signature.

```
trace_accumulated_transit_costs(db_path: str, start_node: int, end_node: int) -> pd.  
    DataFrame
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
ACCUMULATED_PATH_COST_USD: XXXXXX.XX
```

Free to Explore

Compare cheapest route with shortest-distance route.

Task 35: Hierarchy Depth Matrix Scorer

Description. Use a recursive CTE from the root manager to assign a depth to every employee.

Function signature.

```
calculate_hierarchy_depth_matrix(db_path: str) -> pd.DataFrame
```

SQL requirement. Use recursive CTEs where appropriate. Keep recursion depth bounded where the query can otherwise revisit nodes. **Target output format**

```
MAXIMUM_TREE_TOPOLOGY_DEPTH: XX  
TOTAL_RECORDS_MAPPED: XXXX
```

Free to Explore

Group employees by depth and plot the hierarchy shape.

Submission Checklist

- Submit only `solutions.py`.
- Do not hardcode output numbers. Your functions must compute answers from the database.
- Keep all function names and signatures unchanged.
- Keep SQL as explicit query strings inside each function.
- Ensure each function prints the target output line(s) and returns the required object.